

```
In [1]: import numpy as np
arr = np.array([1,2,3,4,5])
print(arr)
```

[1 2 3 4 5]

```
In [2]: a = [1,2,3]
b = [4,5,6]
c = a+b
print(c)
```

[1, 2, 3, 4, 5, 6]

```
In [3]: a = [1,2,3]
b = [4,5,6]
res = []
for i in range(len(a)):
    res.append(a[i]+b[i])
print(res)
```

[5, 7, 9]

```
In [5]: import numpy as np
a = np.array([1,2,3])
b = np.array([4,5,6])
print(a+b)
print(a-b)
print(a*b)
print(a//b)
print(a%b)
print(a**b)
```

[5 7 9]

[-3 -3 -3]

[4 10 18]

[0 0 0]

[1 2 3]

[1 32 729]

```
In [12]: import numpy as np
arr1 = np.array([[1,2,3],[4,5,6]])
arr2 = np.array([[[1,2,3],[4,5,6],[7,8,9]]])
print(arr1)
print(arr2)
print(arr1.shape)
print(arr2.shape)
print(arr1.ndim)
print(arr2.ndim)
print(arr1.size)
print(arr2.size)
print(arr1.dtype)
```

```

[[1 2 3]
 [4 5 6]]
[[[1 2 3]
  [4 5 6]
  [7 8 9]]]
(2, 3)
(1, 3, 3)
2
3
6
9
int64

```

Array creation in numpy

```

In [29]: import numpy as np
a = np.array([1,2,3,4,5])
b = np.zeros(5)
print(a)
print(b)
c = np.zeros((2,2))
print(c)
d = np.ones((2,2))
print(d)
f = np.arange(1,10,2)
print(f)
print(np.linspace(0,20,5))
print(np.random.rand(5))
print(np.random.randint(1,100,20))
print(np.random.randn(5))

```

```

[1 2 3 4 5]
[0. 0. 0. 0. 0.]
[[0. 0.]
 [0. 0.]]
[[1. 1.]
 [1. 1.]]
[1 3 5 7 9]
[ 0.  5. 10. 15. 20.]
[0.76410899 0.13442414 0.67650797 0.35473606 0.44735603]
[19 46 70 21 69 59  6 55 66 16 89 19 59 13 86 55 11 89 16 31]
[ 0.83091466  0.30804854 -0.23028133  0.33484368 -0.03620646]

```

```

In [30]: # Numpy Indexing and Slicing

```

```

In [54]: import numpy as np
a = np.array([5,10,15,20,25])
print(a[2])
b = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(b[1][1])
print(a[:4])
print(a[::2])
print(b[2])
print(b[:,1])
print(b[1,:])
print(b[-2])
print(a[-4])
print(a[:: -2])
print(a[-2::])
print(a[a<20])

```

```

print(a[a>=15])
print(a[[0,4,3]])
print(a[a%2==0])
a[2] = 30
print(a)
b[2] = 11
print(b)
b[2][2] =12
print(b)

```

```

15
5
[ 5 10 15 20]
[ 5 15 25]
[7 8 9]
[2 5 8]
[4 5 6]
[4 5 6]
10
[25 15  5]
[20 25]
[ 5 10 15]
[15 20 25]
[ 5 25 20]
[10 20]
[ 5 10 30 20 25]
[[ 1  2  3]
 [ 4  5  6]
 [11 11 11]]
[[ 1  2  3]
 [ 4  5  6]
 [11 11 12]]

```

Mathematical Operation in numpy

```

In [56]: #scalar Broadcasting
import numpy as np
a = np. array([1,2,3])
print(a+10)
#2d broadcasting
b = np.array([[1,2,3],[2,4,6]])
print(b+10)
#row broadcasting
print(b+a)

```

```

[11 12 13]
[[11 12 13]
 [12 14 16]]
[[2 4 6]
 [3 6 9]]

```

```

In [62]: #Matrix Operation
import numpy as np
a = np.array([[1,2],[3,4]])
b = np.array([[4,3],[2,1]])
print(a*b)
print(np.dot(a,b))
print(a@b)
print(np.matmul(a,b))

```

```
#transpose  
print(a.T)
```

```
[[4 6]  
 [6 4]]  
[[ 8  5]  
 [20 13]]  
[[ 8  5]  
 [20 13]]  
[[ 8  5]  
 [20 13]]  
[[4 6]  
 [6 4]]
```

```
In [2]: #round function  
import numpy as np  
arr = np.array([1.2333,5.666])  
print(np.round(arr,2))  
#floor  
print(np.floor(arr))  
#ceil  
print(np.ceil(arr))
```

```
[1.23 5.67]  
[1.  5.]  
[2.  6.]
```

Statistical Function

```
In [16]: import numpy as np  
a = np.array([10,20,30,40,50])  
print(np.mean(a))  
print(np.median(a))  
print(np.std(a))  
print(np.max(a))  
print(np.min(a))  
print(np.percentile(a,50))  
print(np.var(a))  
print(np.argmin(a))  
print(np.argmax(a))  
b = np.array([1,2,3,4,5])  
c = np.array([10,8,6,4,2])  
print(np.corrcoef(b,c))  
#aggregate Function  
d = np.array([[1,2,3],[4,5,6]])  
print(np.sum(d))  
print(np.sum(d,axis=0))  
print(np.sum(d,axis=1))
```

```

30.0
30.0
14.142135623730951
50
10
30.0
200.0
0
4
[[ 1. -1.]
 [-1.  1.]]
21
[5 7 9]
[ 6 15]

```

```

In [33]: import numpy as np
StudentScore = np.random.randint(1,101,size = (10,5))
print("Student Score:",StudentScore)
print("Average Score:",np.mean(np.sum(StudentScore,axis=1)))
print("Highest Score:",np.max(np.sum(StudentScore,axis=1)))
print("Lowest Score:",np.min(np.sum(StudentScore,axis=1)))
normalized = (StudentScore - StudentScore.min()) / (StudentScore.max() -
print(normalized)

```

```

Student Score: [[77 62 11 40 66]
 [73 21 19 23 54]
 [64 85 40 52 20]
 [73 25 68 36 52]
 [66 77 98  4 47]
 [92 60 36 86 88]
 [46 88 32 98  1]
 [ 2 84 46  5 73]
 [ 3 48 34 48 85]
 [13 64 23  7 59]]
Average Score: 247.4
Highest Score: 362
Lowest Score: 166
[[0.78350515 0.62886598 0.10309278 0.40206186 0.67010309]
 [0.74226804 0.20618557 0.18556701 0.22680412 0.54639175]
 [0.64948454 0.86597938 0.40206186 0.5257732  0.19587629]
 [0.74226804 0.24742268 0.69072165 0.36082474 0.5257732 ]
 [0.67010309 0.78350515 1.          0.03092784 0.4742268 ]
 [0.93814433 0.60824742 0.36082474 0.87628866 0.89690722]
 [0.46391753 0.89690722 0.31958763 1.          0.          ]
 [0.01030928 0.8556701  0.46391753 0.04123711 0.74226804]
 [0.02061856 0.48453608 0.34020619 0.48453608 0.86597938]
 [0.12371134 0.64948454 0.22680412 0.06185567 0.59793814]]

```

```

In [32]: import numpy as np
StudentScore = np.random.randint(1,101,size = (10,5))
print("Student Score:",StudentScore)
print("Average Score:",np.mean(np.sum(StudentScore,axis=0)))
print("Highest Score:",np.max(np.sum(StudentScore,axis=0)))
print("Lowest Score:",np.min(np.sum(StudentScore,axis=0)))

```

```
Student Score: [[ 4 59 77 41 35]
 [69 92  9 42 70]
 [ 8 40 81 45 84]
 [88 28 58 31 76]
 [70 12 29 28 29]
 [77 45 34 44 25]
 [70  6  3 81 94]
 [36 60 59 27 53]
 [84 13 13 84 29]
 [36  2 79 93 50]]
Average Score: 480.4
Highest Score: 545
Lowest Score: 357
```

```
In [35]: import numpy as np
arr = np.array([10,20,30,40,50])
print(arr)
print(np.sum(arr[arr>25]))
```

```
[10 20 30 40 50]
120
```

PANDAS

```
In [9]: #series
import pandas as pd
a = pd.Series([1,2,3,4])
print(a)
print(a.values)
print(a.index)
print(a.dtype)
#accessing elements
print(a[1])
#accessing element using loc
print(a.loc[2])
print(a.iloc[2])
#slicing
print(a[1:3])
print(a.loc[1:3])
print(a.iloc[2:5])
```

```

0    1
1    2
2    3
3    4
dtype: int64
[1 2 3 4]
RangeIndex(start=0, stop=4, step=1)
int64
2
3
3
1    2
2    3
dtype: int64
1    2
2    3
3    4
dtype: int64
2    3
3    4
dtype: int64

```

Create Index in Series

```

In [31]: import pandas as pd
arr = pd.Series([1,2,3,4,5], index = ['A','B','C', 'D', 'E'])
print(arr)
print(arr[['A','C','E']])
print(arr['E'])
#vectorized operation - no loop required
print(arr+5)
print(arr['A']+5)
print(arr)
print(arr.iloc[1]) # based on position
print(arr.loc['A']) # based on index value
print(arr.sum())
print(arr.mean())
print(arr.max())
print(arr.min())
print(arr.std())
print(arr.median())
print(arr[arr%2==0])
#count values
print(len(arr))
print(arr.value_counts())

```

```

A    1
B    2
C    3
D    4
E    5
dtype: int64
A    1
C    3
E    5
dtype: int64
5
A    6
B    7
C    8
D    9
E   10
dtype: int64
6
A    1
B    2
C    3
D    4
E    5
dtype: int64
2
1
15
3.0
5
1
1.5811388300841898
3.0
B    2
D    4
dtype: int64
5
1    1
2    1
3    1
4    1
5    1
Name: count, dtype: int64

```

String Operation

```

In [23]: import pandas as pd
name = pd.Series(['midhun','vishnu','shankar','sakthi'])
print(name)
print(name.str.upper())
print(name.str.len())
print(name.str.capitalize())

```



```

0    midhun
1    vishnu
2    shankar
3    sakthi
dtype: object
0    MIDHUN
1    VISHNU
2    SHANKAR
3    SAKTHI
dtype: object
0    6
1    6
2    7
3    6
dtype: int64
0    Midhun
1    Vishnu
2    Shankar
3    Sakthi
dtype: object

```

Dictionaries in pandas

```

In [27]: import pandas as pd
a = pd.Series({"Midhun":16,"Ram":30,"Akash":1})
print(a)
print(a["Midhun"])

```

```

Midhun    16
Ram       30
Akash      1
dtype: int64
16

```

```

In [30]: import pandas as pd
s1 = pd.Series([10,20,30],index=['A','B','C'])
s2 = pd.Series([1,2,3],index=['B','C','D'])
print(s1+s2)
s = pd.Series([1,2,2,3,4,5,5])
print(s.count())
print(s.unique())
print(s.nunique())

```

```

A    NaN
B    21.0
C    32.0
D    NaN
dtype: float64
7
[1 2 3 4 5]
5

```

Dataframe in Pandas

```

In [29]: import pandas as pd
#method1 : from dictionary
data = { 'name':['Midhun','Nagul','Ram'],'rollno':[16,20,30], 'Age':[24,24,24]}
df=pd.DataFrame(data)
print(df)
#method2 : from series

```

```

s1 = pd.Series([1,2,3,4,5])
s2 = pd.Series([5.6,9.54,9.4,54.6,6])
s3 = pd.Series([True,False,True,False])
df = pd.DataFrame({'sno':s1,'cgpa':s2, 'attendance':s3})
print(df)
#list of list
data = [['Midhun',24,98],['Ram',23,89],['Akash',24,97]]
df=pd.DataFrame(data,columns=["Name","Age","Marks"])
print(df)
#list of dictionary to Data Frame
data=[{"name":"Midhun","course":"Data Science","Grade":"A"},
      {"name":"Madhu","course":"AIML","Grade":"A+"},
      {"name":"Akash","course":"CS","Grade":"A++"}]
df = pd.DataFrame(data, index=["Std1","Std2","Std3"])
print(df)
#adding one extra Columns
df['mark'] = [98,87,67]
print(df)
# adding one row
Std4 = pd.DataFrame([{"name":"Mahi","course":"Data Science","Grade":"0",
df = pd.concat([df,Std4])
print(df)
#create multiple rows
students = pd.DataFrame([{"name":"Ragu","course":"CSDA","Grade":"B","mark"
                        {"name":"Raj","course":"AIML","Grade":"D","mark"
df = pd.concat([df,students])
print(df)
#column Access by bracket notation and dot notation
print(df["name"])
print(df.mark)
print(df[["course","Grade"]])
#shape attribute
print(df.shape)
#find number of columns
print(df.shape[1])
#find number of rows
print(df.shape[0])
# find datatype of all columns
print(df.dtypes)
#info
print(df.info())
#describe
print(df.describe())
#select specific row
print(df.loc[['Std1','Std4']])
print(df.iloc[0])
#updatevalue
df.loc['Std1', 'mark'] = 87
print(df)
#delete row
df = df.drop('Std1')
print(df)
#delete column
df=df.drop(["Grade"],axis=1)
print(df)
#sorting
df.sort_values('mark', inplace=True)
print(df)
df.sort_values('mark', ascending=False, inplace=True)
print(df)

```

```
#statistical Function  
print(df['mark'].max())
```

	name	rollno	Age
0	Midhun	16	24
1	Nagul	20	20
2	Ram	30	23

	sno	cgpa	attendance
0	1	5.60	True
1	2	9.54	False
2	3	9.40	True
3	4	54.60	False
4	5	6.00	NaN

	Name	Age	Marks
0	Midhun	24	98
1	Ram	23	89
2	Akash	24	97

	name	course	Grade
Std1	Midhun	Data Science	A
Std2	Madhu	AIML	A+
Std3	Akash	CS	A++

	name	course	Grade	mark
Std1	Midhun	Data Science	A	98
Std2	Madhu	AIML	A+	87
Std3	Akash	CS	A++	67

	name	course	Grade	mark
Std1	Midhun	Data Science	A	98
Std2	Madhu	AIML	A+	87
Std3	Akash	CS	A++	67
Std4	Mahi	Data Science	0	99

	name	course	Grade	mark
Std1	Midhun	Data Science	A	98
Std2	Madhu	AIML	A+	87
Std3	Akash	CS	A++	67
Std4	Mahi	Data Science	0	99
Std5	Ragu	CSDA	B	50
Std6	Raj	AIML	D	75

Std1	Midhun
Std2	Madhu
Std3	Akash
Std4	Mahi
Std5	Ragu
Std6	Raj

Name: name, dtype: object

Std1	98
Std2	87
Std3	67
Std4	99
Std5	50
Std6	75

Name: mark, dtype: int64

	course	Grade
Std1	Data Science	A
Std2	AIML	A+
Std3	CS	A++
Std4	Data Science	0
Std5	CSDA	B
Std6	AIML	D

(6, 4)

4

6

name object

course object

```

Grade      object
mark       int64
dtype: object
<class 'pandas.core.frame.DataFrame'>
Index: 6 entries, Std1 to Std6
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0    name    6 non-null      object
1   course  6 non-null      object
2   Grade    6 non-null      object
3   mark     6 non-null      int64
dtypes: int64(1), object(3)
memory usage: 240.0+ bytes
None

```

```

           mark
count  6.000000
mean   79.333333
std    19.106718
min    50.000000
25%    69.000000
50%    81.000000
75%    95.250000
max    99.000000

```

```

           name    course  Grade  mark
Std1  Midhun  Data Science    A    98
Std4   Mahi  Data Science    0    99
name           Midhun
course        Data Science
Grade          A
mark           98

```

```

Name: Std1, dtype: object
           name    course  Grade  mark
Std1  Midhun  Data Science    A    87
Std2   Madhu          AIML    A+    87
Std3   Akash           CS    A++   67
Std4   Mahi  Data Science    0    99
Std5   Ragu          CSDA    B    50
Std6   Raj          AIML    D    75

```

```

           name    course  Grade  mark
Std2  Madhu          AIML    A+    87
Std3  Akash           CS    A++   67
Std4  Mahi  Data Science    0    99
Std5  Ragu          CSDA    B    50
Std6  Raj          AIML    D    75

```

```

           name    course  mark
Std2  Madhu          AIML    87
Std3  Akash           CS    67
Std4  Mahi  Data Science    99
Std5  Ragu          CSDA    50
Std6  Raj          AIML    75

```

```

           name    course  mark
Std5  Ragu          CSDA    50
Std3  Akash           CS    67
Std6  Raj          AIML    75
Std2  Madhu          AIML    87
Std4  Mahi  Data Science    99
name           course  mark
Std4  Mahi  Data Science    99
Std2  Madhu          AIML    87

```

Std6	Raj	AIML	75
Std3	Akash	CS	67
Std5	Ragu	CSDA	50
99			

```
In [19]: import pandas as pd
df = pd.read_csv("people-100.csv")
print(df)
print(df.head(5))
print(df.isnull())
print(df.tail(5))
print(df.to_string())
print(df['Sex'])
print(df.isnull().sum())
df[df["Sex"].isnull()]
print(df.columns)
```

	Index	User Id	First Name	Last Name	Sex	\
0	1	88F7B33d2bcf9f5	Shelby	Terrell	Male	
1	2	f90cD3E76f1A9b9	Phillip	Summers	Female	
2	3	DbeAb8CcdfcFC2c	Kristine	Travis	Male	
3	4	A31Bee3c201ef58	Yesenia	Martinez	Male	
4	5	1bA7A3dc874da3c	Lori	Todd	Male	
..	...	...	...	...	...	
95	96	5eFda7caAeB260E	Dennis	Barnes	Female	
96	97	CCbFce93d3720bE	Steve	Patterson	Female	
97	98	2fEc528aFAF0b69	Wesley	Bray	Male	
98	99	Adc7ad9B6e4A1Fe	Summer	Oconnell	Female	
99	100	b8D0ad3490FC7e1	Mariah	Bernard	Male	

	Email	Phone	Date of birth	\
0	elijah57@example.net	001-084-906-7849x73518	1945-10-26	
1	bethany14@example.com	214.112.6044x4913	1910-03-24	
2	bthompson@example.com	277.609.7938	1992-07-02	
3	kaitlinkaiser@example.com	584.094.6111	2017-08-03	
4	buchananmanuel@example.net	689-207-3558x7233	1938-12-01	
..	...	...	...	
95	bmartin@example.org	001-095-524-2112x257	1954-07-30	
96	latasha46@example.net	001-865-478-5157	1932-04-29	
97	reginal1@example.org	995-542-3004x76800	1994-12-28	
98	alexiscantrell@example.org	001-273-685-6932x092	2012-04-12	
99	pcopeland@example.org	(341)594-6554x44657	2016-11-15	

	Job Title
0	Games developer
1	Phytotherapist
2	Homeopath
3	Market researcher
4	Veterinary surgeon
..	...
95	Software engineer
96	Barrister
97	Police officer
98	Broadcast journalist
99	IT sales professional

[100 rows x 9 columns]

	Index	User Id	First Name	Last Name	Sex	\
0	1	88F7B33d2bcf9f5	Shelby	Terrell	Male	
1	2	f90cD3E76f1A9b9	Phillip	Summers	Female	
2	3	DbeAb8CcdfcFC2c	Kristine	Travis	Male	
3	4	A31Bee3c201ef58	Yesenia	Martinez	Male	
4	5	1bA7A3dc874da3c	Lori	Todd	Male	

	Email	Phone	Date of birth	\
0	elijah57@example.net	001-084-906-7849x73518	1945-10-26	
1	bethany14@example.com	214.112.6044x4913	1910-03-24	
2	bthompson@example.com	277.609.7938	1992-07-02	
3	kaitlinkaiser@example.com	584.094.6111	2017-08-03	
4	buchananmanuel@example.net	689-207-3558x7233	1938-12-01	

	Job Title
0	Games developer
1	Phytotherapist
2	Homeopath
3	Market researcher
4	Veterinary surgeon

	Index	User Id	First Name	Last Name	Sex	Email	Phone	Date of birth
0	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False
..	...	...	...	...	...	...	...	...
95	False	False	False	False	False	False	False	False
96	False	False	False	False	False	False	False	False
97	False	False	False	False	False	False	False	False
98	False	False	False	False	False	False	False	False
99	False	False	False	False	False	False	False	False

	Job Title
0	False
1	False
2	False
3	False
4	False
..	...
95	False
96	False
97	False
98	False
99	False

[100 rows x 9 columns]

	Index	User Id	First Name	Last Name	Sex
95	96	5eFda7caAeB260E	Dennis	Barnes	Female
96	97	CCbFce93d3720bE	Steve	Patterson	Female
97	98	2fEc528aFAF0b69	Wesley	Bray	Male
98	99	Adc7ad9B6e4A1Fe	Summer	Oconnell	Female
99	100	b8D0aD3490FC7e1	Mariah	Bernard	Male

	Email	Phone	Date of birth
95	bmartin@example.org	001-095-524-2112x257	1954-07-30
96	latasha46@example.net	001-865-478-5157	1932-04-29
97	regina11@example.org	995-542-3004x76800	1994-12-28
98	alexiscantrell@example.org	001-273-685-6932x092	2012-04-12
99	pcopeland@example.org	(341)594-6554x44657	2016-11-15

	Job Title
95	Software engineer
96	Barrister
97	Police officer
98	Broadcast journalist
99	IT sales professional

Index	User Id	First Name	Last Name	Sex
-------	---------	------------	-----------	-----

Email	Phone	Date of birth			
0 1 88F7B33d2bcf9f5 57@example.net	Shelby 001-084-906-7849x73518	Terrell 1945-10-26	Male	elijah	Games developer
1 2 f90cD3E76f1A9b9 14@example.com	Phillip 214.112.6044x4913	Summers 1910-03-24	Female	bethany	Phytotherapist
2 3 DbeAb8CcdfeFC2c on@example.com	Kristine 277.609.7938	Travis 1992-07-02	Male	bthomps	Homeopath
3 4 A31Bee3c201ef58 er@example.com	Yesenia 584.094.6111	Martinez 2017-08-03	Male	kaitlinkais	Market researcher
4 5 1bA7A3dc874da3c el@example.net	Lori 689-207-3558x7233	Todd 1938-12-01	Male	buchananmanu	Veterinary surgeon
5 6 bfDD7CDEF5D865B er@example.org	Erin 001-171-649-9856x5553	Day 2015-10-28	Male	tconn	Waste management officer
6 7 bE9EEf34cB72AF7 an@example.com	Katherine +1-773-151-6685x49162	Buck 1989-01-22	Female	conniecow	Intelligence analyst
7 8 2EFC6A4e77FaEaC op@example.com	Ricardo 001-447-699-7998x88612	Hinton 1924-03-26	Male	wyattbish	Hydrogeologist
8 9 baDcC4DeefD8dEB nn@example.net	Dave 603-428-2429x27392	Farrell 2018-10-06	Male	nmcca	Lawyer
9 10 8e4FB470FE19bF0 ll@example.org	Isaiah +1-511-372-1544x8206	Downs 1964-09-20	Male	virginiaterre	Engineer, site
10 11 BF0BbA03C29Bb3b hy@example.com	Sheila 895.881.4746	Ross 2008-03-20	Female	huangcat	Advertising account executive
11 12 F738c69fB34E62E oy@example.org	Stacy 710.673.3213x80335	Newton 1980-10-20	Male	rayler	Warden/ranger
12 13 C03fDADdAadAdCe le@example.org	Mandy (992)466-1305x4947	Blake 2007-12-08	Male	jefferynob	(histocompatibility and immunogenetics)
13 14 b759b74BD1dE80d 44@example.com	Bridget (216)627-8359	Nash 2004-06-28	Female	mercedes	Social worker
14 15 1F0B7D65A00DAF9 da@example.org	Crystal +1-024-377-5391	Farmer 1992-03-09	Male	pmiran	Agricultural consultant
15 16 50Bb061cB30B461 la@example.net	Thomas +1-360-880-0766	Knight 2006-02-18	Female	braunpriscil	Sport and exercise psychologist
16 17 D6dbA5308fEC4BC ks@example.com	Maurice (246)187-4969	Rangel 2004-08-20	Male	sheenaban	Secretary/administrator
17 18 311D775990f066d er@example.org	Frank 429.965.3902x4447	Meadows 2008-09-16	Male	gbrew	Audiological scientist
18 19 7F7E1BAcb0C9AFf on@example.com	Alvin 219.436.0887x07551	Paul 1949-05-12	Male	gilbertdonalds	Teacher, adult education
19 20 88473e15D5c3cD0	Jared	Mitchell	Female	jcort	

ez@example.com	+1-958-849-6781	1921-01-18	
Paediatric nurse			
20 21 b31D271F8c200AB Jacqueline	Norton Female	cari	
as@example.net 819.309.7679x59173	1952-10-09		
Scientist, marine			
21 22 42F4BdA841aBadC Colleen	Hatfield Female	fkn	
ox@example.org 638.584.1090	1949-10-14		
Commercial horticulturist			
22 23 cBbBcA0FCA3C4Bc Randy	Barnes Male	huangbi	
ll@example.org 001-960-629-7164x67214	1947-12-30		
Outdoor activities/education manager			
23 24 f1f89173353aD90 Janice	Rhodes Female	juarezdominiq	
ue@example.net 001-249-314-9742x6996	1999-11-01		
Drilling engineer			
24 25 c5B09fb33e8bA0A Alfred	Mcneil Female	cassandramorr	
is@example.com (468)276-9509x53058	1993-05-28		
Systems analyst			
25 26 c9F2282C40BEC1E Sean	Levine Male	sallymill	
er@example.net 4915828504	2010-10-09		
Conservation officer, nature			
26 27 9c1bc7EC53Fb7cE Louis	Payne Male	bsulliv	
an@example.net 6232695307	1916-01-29		
Counsellor			
27 28 ddEc50e2A2e3a2B Brittney	Vega Female	ayalajo	
se@example.net 945-739-8686	1932-10-31		
Recycling officer			
28 29 66F096D36Ebae11 Judy	Buckley Male	irosal	
es@example.net 001-654-208-1241x52830	1963-07-28		
Art gallery manager			
29 30 F0fE2faAd78F8b5 Norman	Weber Male	mconr	
ad@example.com 223.002.0429	1957-05-21		
Gaffer			
30 31 5d2feAfbCAA6B5 Isaiah	Camacho Female	jimbla	
ke@example.org 001-536-544-3367	1966-04-07		
Food technologist			
31 32 cDa5F303fCd6dEa Jacqueline	Gallagher Male	nsamps	
on@example.net (247)762-8934	1999-02-25		
Building services engineer			
32 33 8Ef7DBfcaB02b6B Bonnie	Andrews Female	caitlin	
24@example.net +1-253-987-2776x9161	1953-12-21		
Seismic interpreter			
33 34 6Dec5b5542F8ed8 Brandon	Schmidt Female	mconl	
ey@example.net +1-386-673-1465x006	1931-05-12		
Engineer, biomedical			
34 35 3Fb8a7f68e12784 Jackson	Sparks Female	reynoldsdarr	
yl@example.net (137)908-3129x65035	1980-11-18		
Set designer			
35 36 035eff50B9A0F24 Melody	Cook Male	jeannov	
ak@example.org (826)792-7381	1963-06-25		
Research scientist (life sciences)			
36 37 aa614aAE4B7Cf0C Leonard	Hurst Male	clinton	
78@example.org 941-038-0427x38800	1938-03-13		
Accountant, chartered management			
37 38 ACcde95AAe3e6cC Gene	Rich Female	luisdele	
on@example.org +1-356-818-6604x89537	1946-08-22		
Surveyor, quantity			
38 39 b6a35de5CB6fc25 Cynthia	Wiggins Female	rosarioda	
ve@example.org (110)858-2437x70190	1984-01-27		
Outdoor activities/education manager			
39 40 e92A191E345fA3A Tanya	Mckinney Female	vickihoust	

on@example.com	(830)774-9002x086	2003-03-12	
Information systems manager			
40 41 7D0AcBF6CCac3fd	Matthew	Stone	Female evelyn
31@example.org	952-381-6360	2017-08-23	Scientist, clinical
(histocompatibility and immunogenetics)			
41 42 CEFA7BBCEf013AE	Kirk	Walsh	Female stephenfull
er@example.org	001-826-496-5529x8661	2009-04-08	
Accounting technician			
42 43 9edBC94aE7cA22a	Willie	Vang	Female haleymathe
ws@example.net	741.168.6854x067	1978-02-02	
Management consultant			
43 44 fFe7BAA737aDbe2	Miguel	Hill	Female tyrone
56@example.org	5247842945	1930-08-26	
Make			
44 45 5F2f3fAca8B0946	Darren	Andrews	Male lhernand
ez@example.com	(975)799-4261	1997-10-04	
Retail banker			
45 46 6bFcf3cc1BC6B4	Haley	Pugh	Female molly
03@example.org	(746)182-6137x2453	1980-09-16	
Commissioning editor			
46 47 f3BD2cBF7eEb6df	Danielle	Estrada	Female jva
ng@example.org	(890)374-9518x772	1930-07-09	
Accountant, chartered management			
47 48 Ee4eB129dC7913A	Becky	Brady	Male erikmuell
er@example.org	(390)002-0863	1957-06-27	
Seismic interpreter			
48 49 dBCEf340C3657Eb	Caitlyn	Frey	Male rivasdominiq
ue@example.org	805-021-3965x46344	1968-01-26	
Jewellery designer			
49 50 E47FB71DD9ACCd9	Joshua	Sweeney	Male daisymcg
ee@example.net	875.994.2100x535	1954-07-28	
Education officer, museum			
50 51 eA3fDd79BE9f0E7	Heidi	Escobar	Female staffordtrav
is@example.net	601-155-3065x1131	1931-09-25	
Estate manager/land agent			
51 52 aF0eE4547Bc025c	Brian	Oconnell	Female saralo
ng@example.net	952-283-1423x733	1911-10-23	
Physiotherapist			
52 53 9F5DeD7aD228F5a	Beverly	Esparza	Female iphel
ps@example.net	+1-327-578-8754x6771	1930-12-09	
Passenger transport manager			
53 54 D3Fa0220dDE4d36	Nathaniel	Rivas	Female roberto
29@example.com	(655)887-2040x37888	1908-11-17	
Call centre manager			
54 55 60FdBFd5e7BE8fF	Debra	Payne	Female yolanda
07@example.org	001-731-525-8400x52593	1927-08-20	
Special educational needs teacher			
55 56 D8bF5Ab2b98caff	Mackenzie	Rocha	Female abbottvet
te@example.net	4225525458	1980-10-21	
Museum/gallery exhibitions officer			
56 57 CD8d33aA25bc8BB	Courtney	Watkins	Female ocha
ng@example.org	210.683.2761x5883	2003-12-07	
Pension scheme manager			
57 58 Fac3BFf0A3d03c	Fred	Olsen	Female amyanders
on@example.com	497-774-3053	1910-04-10	
Archaeologist			
58 59 e552D7ddafe1FFb	Ryan	Nelson	Female qnorm
an@example.org	956.330.2951	1924-05-02	Historic b
uildings inspector/conservation officer			
59 60 0f8deedb629A5f6	Grace	Phelps	Male clarkeange

la@example.net	(034)867-8827x6777	1909-10-15		
Petroleum engineer				
60 61 bB9e49E506F65ed	Shari	Daugherty	Male	kalvara
do@example.org	001-951-655-4798x6124	1944-11-24		
Curator				
61 62 Ed724605A403D91	Kelli	Garner	Male	jodyvince
nt@example.org	995.000.4213x0982	2010-01-17		
Retail banker				
62 63 0aBE5ACb18E0c10	Jackie	Bennett	Male	hutchinsonki
rk@example.com	001-740-937-0846x0087	1915-11-11		
Neurosurgeon				
63 64 5D2cb63CaAF53f6	Leslie	Conway	Female	floreschristi
na@example.org	795.782.4384x555	1983-11-06		
Chiropractor				
64 65 Ee6974f90eeCe18	Harold	Barnett	Female	nathan
65@example.org	+1-026-265-6392	1943-03-15		
Biochemist, clinical				
65 66 cEf02C076afa07f	Larry	Harper	Male	maria
32@example.org	+1-244-630-3792x4121	2021-05-05		
Scientist, water quality				
66 67 9Df5Ba591bF3EFf	Mike	Ward	Female	imccullou
gh@example.com	116-729-5046	1967-11-09		
Hydrologist				
67 68 3faB1CBfEFBDdD4	Brittney	Rubio	Female	corey
92@example.com	593.976.2528	1959-12-24		
Biochemist, clinical				
68 69 Ebcefdf75eCb0a9	Frank	Pineda	Male	daltoncalv
in@example.net	(035)961-5060x9182	1926-03-10		
Hospital pharmacist				
69 70 e75e5DBfcb68887	Sandra	Wu	Male	uban
ks@example.com	+1-096-606-6454x067	1925-04-28		
Warehouse manager				
70 71 6a53a8D41dDF6de	Ryan	Benton	Male	lopezdebb
ie@example.org	+1-695-557-9948x485	2020-10-06		
Physiological scientist				
71 72 F0d3bD1aaf9E3Bc	Tamara	Hull	Male	meagan
39@example.net	017.665.3744x7944	1933-01-31		
English as a second language teacher				
72 73 5bC87340799FBD0	Jean	Ritter	Female	kristina
76@example.com	(954)060-1066	1985-08-06		
Financial trader				
73 74 dBfA17Aaf16b4ab	Veronica	Briggs	Female	weissbridg
et@example.com	+1-521-589-2387x48490	1974-06-08		
Structural engineer				
74 75 c935b7Eb6FA0B0F	Kim	Andrews	Female	wpeters
en@example.org	7677125383	1990-11-15		
Biochemist, clinical				
75 76 b3e15e65Ca2CcBf	Tina	Cunningham	Male	wongma
ry@example.org	079-907-5051	1956-11-29		
Race relations officer				
76 77 dade3452F0c32FD	Jonathon	Atkinson	Male	gailfren
ch@example.net	874-037-2032x932	2011-07-19		
Psychologist, forensic				
77 78 AdEd6cfD85DeC46	Jermaine	Reid	Female	vpa
ul@example.com	(742)214-8691	1974-08-18		
Newspaper journalist				
78 79 DAF111987098ae4	Regina	Stevens	Male	xpoo
le@example.net	891-359-2684	2011-11-28		
Public house manager				
79 80 6e6a5b885F6496d	Terrence	Huff	Male	cassandra

80@example.org	221.800.6408x5416	1944-02-27		
Careers information officer				
80 81 12DCb4ED8E01D5C	Tyler	Foley	Female	johnathan
72@example.org 001-386-469-3075x8030		1908-09-19		
Economist				
81 82 ElcB5cA8CA7CC0a	Andrew	Waters	Male	nha
ll@example.net +1-376-865-2765x3351		1948-05-14		
Jewellery designer				
82 83 AedDfaE8Cf49F07	Reginald	Stephenson	Male	erikaba
ll@example.net +1-832-500-6044x475		2010-02-08		
Contracting civil engineer				
83 84 bff9853aFAeF772	Douglas	Reese	Female	nixonvanes
sa@example.net 001-834-660-8312x9864		1961-11-11		
Higher education lecturer				
84 85 E883773cA5219Be	Helen	Williamson	Female	melvin
08@example.net 001-377-726-4229		1911-08-11		
Lecturer, further education				
85 86 CB19EafEbBf9eC	Mario	Vaughn	Male	obla
ke@example.com 160-144-5039x12276		1990-07-08		
Research scientist (life sciences)				
86 87 5834700fbEd2771	Chelsea	Dickson	Male	johnnyhendric
ks@example.net 001-698-651-0138x18588		1958-05-13		
Teacher, early years/pre				
87 88 2b0Ab1Dc9E01D7E	Dustin	Bailey	Male	pbarr
on@example.net +1-965-621-1157x345		1908-08-22		
Travel agency manager				
88 89 3f3a3D89ad042Dd	Harry	Medina	Female	olsenmal
ik@example.net +1-451-099-5805		1947-08-24		
Technical sales engineer				
89 90 9425E2F38C408ef	Kathy	Haney	Female	teresa
37@example.com (164)105-8456		1955-09-02		
Charity fundraiser				
90 91 F0aeC9c2759F3C6	Alison	Nixon	Female	zmil
es@example.net 3506680871		1941-07-10		
Patent attorney				
91 92 d6EA619A7C4aA95	Jamie	Hardy	Female	sheenadoug
as@example.com (900)803-9295x11533		1994-07-17		
Conservator, furniture				
92 93 2A33E7Cad1bb0F5	Melody	Cox	Female	evan
90@example.org (626)520-5080x3511		1974-07-30		
Dance movement psychotherapist				
93 94 d181FFB7d3E68bb	Xavier	Cole	Male	nicolas
90@example.org 8164259975		1938-11-29		
Financial planner				
94 95 feaBf8dAE0C0d6F	Dillon	Guzman	Female	angelanavar
ro@example.net 971-992-4521		1942-04-01		
Air broker				
95 96 5eFda7caAeB260E	Dennis	Barnes	Female	bmart
in@example.org 001-095-524-2112x257		1954-07-30		
Software engineer				
96 97 CCbFce93d3720bE	Steve	Patterson	Female	latasha
46@example.net 001-865-478-5157		1932-04-29		
Barrister				
97 98 2fEc528aFAF0b69	Wesley	Bray	Male	regina
11@example.org 995-542-3004x76800		1994-12-28		
Police officer				
98 99 Adc7ad9B6e4A1Fe	Summer	Oconnell	Female	alexiscantre
ll@example.org 001-273-685-6932x092		2012-04-12		
Broadcast journalist				
99 100 b8D0aD3490FC7e1	Mariah	Bernard	Male	pcopela

```

nd@example.org      (341)594-6554x44657      2016-11-15
IT sales professional
0      Male
1      Female
2      Male
3      Male
4      Male
...
95     Female
96     Female
97      Male
98     Female
99      Male
Name: Sex, Length: 100, dtype: object
Index      0
User Id    0
First Name 0
Last Name  0
Sex        0
Email      0
Phone      0
Date of birth 0
Job Title  0
dtype: int64
Index(['Index', 'User Id', 'First Name', 'Last Name', 'Sex', 'Email', 'Phone',
      'Date of birth', 'Job Title'],
      dtype='object')

```

```

In [32]: import pandas as pd
df = pd.read_json("titanic.json")
print(df)
df.head()
print(df.isnull())
print(df.isnull().sum())
df[df['Age'].isnull()]
mean_age = df['Age'].mean()
df['Age'] = df['Age'].fillna(mean_age)
print(df.isnull().sum())
mode_embarked = df['Embarked'].mode()[0]
df['Embarked'] = df['Embarked'].fillna(mode_embarked)
print(df.isnull().sum())
df.dropna()
df.dropna(axis=1)
print(df.columns)
df['Cabin'] = df['Cabin'].fillna("General")
print(df.isnull().sum())
print(df.groupby('Pclass')['Fare'].mean())
df.groupby('Sex')['Survived'].sum()
print(df.groupby('Pclass').agg({
    'Age': ['mean', 'min', 'max'],
    'Fare': ['mean', 'sum']
})))
#transform
df['Avg_Class_Fare'] = df.groupby('Pclass')['Fare'].transform('mean')
print(df)
print(df.columns)

```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	
..	...	...	...	
886	887	0	2	
887	888	1	1	
888	889	0	3	
889	890	1	1	
890	891	0	3	

p	\	Name	Sex	Age	SibS
0		Braund, Mr. Owen Harris	male	22.0	
1					
1		Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	
1					
2		Heikkinen, Miss. Laina	female	26.0	
0					
3		Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	
1					
4		Allen, Mr. William Henry	male	35.0	
0					
..		...	...	...	
...					
886		Montvila, Rev. Juozas	male	27.0	
0					
887		Graham, Miss. Margaret Edith	female	19.0	
0					
888		Johnston, Miss. Catherine Helen "Carrie"	female	NaN	
1					
889		Behr, Mr. Karl Howell	male	26.0	
0					
890		Dooley, Mr. Patrick	male	32.0	
0					

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	None	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	None	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	None	S
...	...	...	...	...	...
886	0	211536	13.0000	None	S
887	0	112053	30.0000	B42	S
888	2	W./C. 6607	23.4500	None	S
889	0	111369	30.0000	C148	C
890	0	370376	7.7500	None	Q

```
[891 rows x 12 columns]
```

[illegible]

3	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...
886	False	False	False	False	False	False	False	False	False	False
887	False	False	False	False	False	False	False	False	False	False
888	False	False	False	False	False	True	False	False	False	False
889	False	False	False	False	False	False	False	False	False	False
890	False	False	False	False	False	False	False	False	False	False

	Fare	Cabin	Embarked
0	False	True	False
1	False	False	False
2	False	True	False
3	False	False	False
4	False	True	False
...	...	...	...
886	False	True	False
887	False	False	False
888	False	True	False
889	False	False	False
890	False	True	False

[891 rows x 12 columns]

PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2
dtype: int64	
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	0
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2
dtype: int64	
PassengerId	0
Survived	0
Pclass	0


```

Name      0
Sex       0
Age       0
SibSp     0
Parch     0
Ticket    0
Fare      0
Cabin     687
Embarked  0
dtype: int64
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
      'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
PassengerId  0
Survived     0
Pclass       0
Name         0
Sex          0
Age          0
SibSp        0
Parch        0
Ticket       0
Fare         0
Cabin        0
Embarked     0
dtype: int64
Pclass
1      84.154687
2      20.662183
3      13.675550
Name: Fare, dtype: float64

```

	Age			Fare	
	mean	min	max	mean	sum
Pclass					
1	37.048118	0.92	80.0	84.154687	18177.4125
2	29.866958	0.67	70.0	20.662183	3801.8417
3	26.403259	0.42	74.0	13.675550	6714.6951

```

PassengerId  Survived  Pclass  \
0            1         0        3
1            2         1        1
2            3         1        3
3            4         1        1
4            5         0        3
..          ...      ...      ...
886          887         0        2
887          888         1        1
888          889         0        3
889          890         1        1
890          891         0        3

```

	Name	Sex	Age
\			
0	Braund, Mr. Owen Harris	male	22.000000
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.000000
2	Heikkinen, Miss. Laina	female	26.000000
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.000000
4	Allen, Mr. William Henry	male	35.000000
..	...	...	...
886	Montvila, Rev. Juozas	male	27.000000
887	Graham, Miss. Margaret Edith	female	19.000000

888	Johnston, Miss. Catherine Helen "Carrie"	female	29.699118
889	Behr, Mr. Karl Howell	male	26.000000
890	Dooley, Mr. Patrick	male	32.000000

	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Avg_Class_Fare
0	1	0	A/5 21171	7.2500	General	S	13.67
5550							
1	1	0	PC 17599	71.2833	C85	C	84.15
4687							
2	0	0	STON/O2. 3101282	7.9250	General	S	13.67
5550							
3	1	0	113803	53.1000	C123	S	84.15
4687							
4	0	0	373450	8.0500	General	S	13.67
5550							
...	...	...	...	...	...	...	
...							
886	0	0	211536	13.0000	General	S	20.66
2183							
887	0	0	112053	30.0000	B42	S	84.15
4687							
888	1	2	W./C. 6607	23.4500	General	S	13.67
5550							
889	0	0	111369	30.0000	C148	C	84.15
4687							
890	0	0	370376	7.7500	General	Q	13.67
5550							

[891 rows x 13 columns]

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
      'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked', 'Avg_Class_Fare'],
      dtype='object')
```

```
In [10]: import pandas as pd
df = pd.read_excel("Supermarket-Sales-Sample-Data.xlsx")
df.tail(10)
```

Out[10]:

	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6
67	NaN	1061	2024-01-31 00:00:00	Daniel Hernandez	2024-02-02 00:00:00	149.99	...
68	NaN	1062	2024-01-31 00:00:00	Jennifer Davis	2024-02-04 00:00:00	69.99	...
69	NaN	1063	2024-02-01 00:00:00	Michael Martinez	2024-02-02 00:00:00	39.99	...
70	NaN	1064	2024-02-01 00:00:00	Jessica Wilson	2024-02-05 00:00:00	199.99	...
71	NaN	1065	2024-02-01 00:00:00	David Rodriguez	2024-02-03 00:00:00	29.99	...
72	NaN	1066	2024-02-02 00:00:00	Sarah Gonzalez	2024-02-06 00:00:00	79.99	...
73	NaN	1067	2024-02-03 00:00:00	Matthew Smith	2024-02-06 00:00:00	49.99	...
74	NaN	1068	2024-02-04 00:00:00	Emily Johnson	2024-02-05 00:00:00	129.99	...
75	NaN	1069	2024-02-04 00:00:00	Daniel Brown	2024-02-08 00:00:00	19.99	...
76	NaN	1070	2024-02-04 00:00:00	Jennifer Hernandez	2024-02-07 00:00:00	149.99	...

In [12]:

!pip install pandas openpyxl

Requirement already satisfied: pandas in c:\users\hp\anaconda3\lib\site-packages (2.3.3)
Requirement already satisfied: openpyxl in c:\users\hp\anaconda3\lib\site-packages (3.1.5)
Requirement already satisfied: numpy>=1.26.0 in c:\users\hp\anaconda3\lib\site-packages (from pandas) (2.3.5)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\hp\anaconda3\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\hp\anaconda3\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\hp\anaconda3\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: et-xmlfile in c:\users\hp\anaconda3\lib\site-packages (from openpyxl) (2.0.0)
Requirement already satisfied: six>=1.5 in c:\users\hp\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

In []: